

Sprawozdanie 3

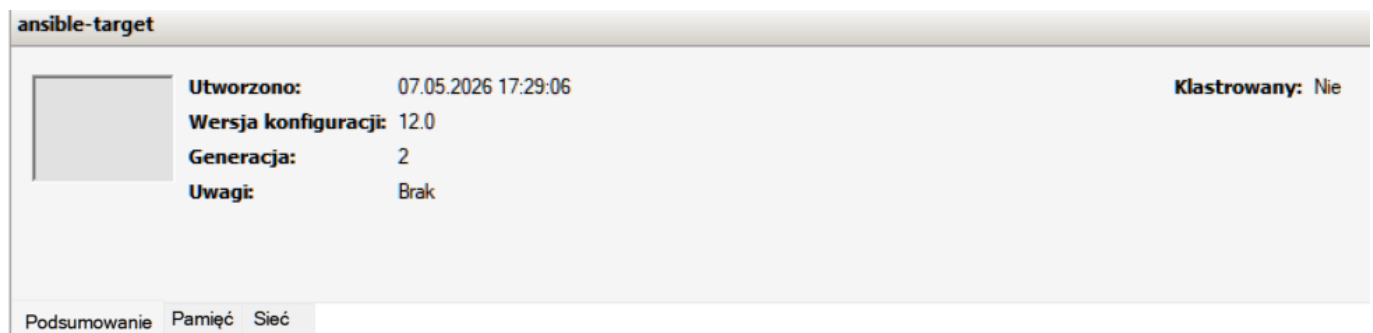
Bartosz Bodulski

gr. 1, Tematy 8-11

Temat 8

Cel zajęć - Automatyzacja i zdalne wykonywanie poleceń za pomocą Ansible

W ramach zajęć musiałem stworzyć drugą maszynę wirtualną, na której automatyzowałbym polecenia za pomocą oprogramowania Ansible - maszyna musi mieć conajmniej 2048 MB RAM oraz 2 logiczne rdzenie:



Potem modyfikuję nazwę mojej początkowej maszyny za pomocą polecenia `hostnamectl set-hostname ansible-master`, aby nadać jej odpowiednią nazwę do poprawnej procedury inwentaryzacji w pliku `inventory.ini`:

```
[Orchestrators]
ansible-master ansible_connection=local

[Endpoints]
ansible-target ansible_host=ansible-target.local ansible_user=ansible
```

Naszym "konduktorem" będzie maszyna `ansible-master`, natomiast odbiorcą rozkazów będzie `ansible-target`! Do komunikacji między maszynami za pomocą dns użyłem Avahi - programu, który pozwala na komunikację między maszynami za pomocą multicast dns: [https://pl.wikipedia.org/wiki/Avahi_\(program\)](https://pl.wikipedia.org/wiki/Avahi_(program))

Testuję połączenie za pomocą wbudowanego polecenia `ping`:

```
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab8$ ansible all -i inventory.ini -m ping
ansible-master | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ansible-target | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

Na podstawie inwentaryzacji piszę pierwszego playbooka - jest to plik z listą kroków, które mają być wykonane przez maszynę docelową. Kopiuje odpowiednie pliki, włączam usługę sshd/rngd i aktualizuje pakiety systemowe:

```
name: Podstawowa konfiguracja i testy systemu
hosts: Endpoints
become: yes # (wymagane do apt i systemd)

tasks:
  - name: ping (wbudowany moduł)
    ansible.builtin.ping:

  - name: Skopiuj plik inwentaryzacji na maszyny Endpoints
    ansible.builtin.copy:
      src: ./inventory.ini
      dest: /tmp/inventory.ini

  - name: Zaktualizuj pakiety w systemie (troche czasu zajmie)
    ansible.builtin.apt:
      upgrade: yes
      update_cache: yes

  - name: Zrestartuj usługę sshd
    ansible.builtin.service:
      name: ssh
      state: restarted

# rngd (RNG tools), jeżeli nie ma to dodajemy ignore_errors
- name: Zrestartuj usługę rngd (jeśli istnieje)
  ansible.builtin.service:
    name: rng-tools
    state: restarted
  ignore_errors: yes
```

Sprawdzamy działanie playbooka - jak widać, krok z rngd zwraca błąd, gdyż nie jest zainstalowany na systemie, ale przerywa działa playbooka. Na koniec dostajemy statystki, które liczą statusy poszczególnych kroków operacji.

```

● bartosz123@ubuntu-devops:~/MD02026s_ITE/ITE/GCL1/BB419678/lab8$ ansible-playbook -i inventory.ini playbook.yml

PLAY [Podstawowa konfiguracja i testy systemu] *****

TASK [Gathering Facts] *****
ok: [ansible-target]

TASK [ping (wbudowany modul)] *****
ok: [ansible-target]

TASK [Skopiuj plik inwentaryzacji na maszyny Endpoints] *****
changed: [ansible-target]

TASK [Zaktualizuj pakiety w systemie (troche czasu zajmie)] *****
ok: [ansible-target]

TASK [Zrestartuj usluge sshd] *****
changed: [ansible-target]

TASK [Zrestartuj usluge rngd (jezeli istnieje)] *****
fatal: [ansible-target]: FAILED! => {"changed": false, "msg": "Could not find the requested service rng-tools: host"}
...ignoring

PLAY RECAP *****
ansible-target      : ok=6   changed=2   unreachable=0   failed=0   skipped=0   rescued=0   ignored=1

```

Jeżeli wykonamy to polecenie jeszcze raz - kopiowanie pliku inwentaryzacji do maszyny docelowej będzie mieć status "unchanged", co ma sens, gdyż ten plik o niezmienionej zawartości już znajdował się na ansible-target po pierwszym wykonaniu polecenia.

```

● bartosz123@ubuntu-devops:~/MD02026s_ITE/ITE/GCL1/BB419678/lab8$ ansible-playbook -i inventory.ini playbook.yml

PLAY [Podstawowa konfiguracja i testy systemu] *****

TASK [Gathering Facts] *****
ok: [ansible-target]

TASK [ping (wbudowany modul)] *****
ok: [ansible-target]

TASK [Skopiuj plik inwentaryzacji na maszyny Endpoints] *****
ok: [ansible-target]

TASK [Zaktualizuj pakiety w systemie (troche czasu zajmie)] *****
ok: [ansible-target]

TASK [Zrestartuj usluge sshd] *****
changed: [ansible-target]

TASK [Zrestartuj usluge rngd (jezeli istnieje)] *****
fatal: [ansible-target]: FAILED! => {"changed": false, "msg": "Could not find the requested service rng-tools: host"}
...ignoring

PLAY RECAP *****
ansible-target      : ok=6   changed=1   unreachable=0   failed=0   skipped=0   rescued=0   ignored=1

```

jeżeli natomiast wyłączymy kartę sieciową - dostaniemy komunikat błędu z statusem "unreachable". Tutaj żadnych zadań nieudało się wykonać.

```

● bartosz123@ubuntu-devops:~/MD02026s_ITE/ITE/GCL1/BB419678/lab8$ ansible-playbook -i inventory.ini playbook.yml

PLAY [Podstawowa konfiguracja i testy systemu] *****

TASK [Gathering Facts] *****
fatal: [ansible-target]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: ssh: connect to host ansible-target port 22: No route to host", "unreachable": true}

PLAY RECAP *****
ansible-target      : ok=0   changed=0   unreachable=1   failed=0   skipped=0   rescued=0   ignored=0

```

Następnie tworzę oddzielny playbook do sprawdzenia działania wcześniej utworzonego artefaktu neovima od Jenkinsa. Sprawdzamy najpierw czy nasza maszyna spełnia wymagania, instalujemy dockera, sprawdzamy działanie w tle, następnie tworzymy kontener, odpalamy oprogramowanie w celu sprawdzenia działa i na koniec wszystko sprzątamy:

```
name: Wdrożenie artefaktu binarnego (Neovim DEB)
hosts: Endpoints
become: yes # Uruchamiaj zadania jako root (docker)
vars:
  artifact_name: "neovim-final.deb"
  deploy_dir: "/opt/neovim_deploy"

tasks:
  # -----
  # Sanity check (z ignore_errors)
  # -----
  - name: Przeprowadź sanity check (czy to Ubuntu i czy ma minimum 512MB
    RAM)
    ansible.builtin.assert:
      that:
        - ansible_facts['distribution'] == "Ubuntu"
        - ansible_facts['memtotal_mb'] >= 512
      fail_msg: "Maszyna nie spełnia wymagań sprzętowych lub
systemowych!"
      success_msg: "Sanity check zakończony sukcesem."
      ignore_errors: yes # "nie ulegaj awarii w przypadku niepowodzenia"

  # -----
  # Sprawdzenie dockera
  # -----
  - name: Zainstaluj Dockera (upewnij się wprost, że istnieje)
    ansible.builtin.apt:
      name: docker.io
      state: present
      update_cache: yes

  - name: Upewnij się, że usługa Docker jest uruchomiona
    ansible.builtin.service:
      name: docker
      state: started
      enabled: yes

  # -----
  # Przesył paczki .deb na ansible-target
  # -----
  - name: Utwórz katalog wdrożeniowy na maszynie docelowej
    ansible.builtin.file:
      path: "{{ deploy_dir }}"
      state: directory
      mode: '0755'

  - name: Skopiuj paczkę DEB z Orchestratora na maszynę docelową
```

```

ansible.builtin.copy:
  src: "./{{ artifact_name }}"
  dest: "{{ deploy_dir }}/{{ artifact_name }}"

# -----
# Uruchamiamy paczkę na kontenerze
# -----
- name: Utwórz Dockerfile, by umieścić/udostępnić plik w kontenerze
  ansible.builtin.copy:
    dest: "{{ deploy_dir }}/Dockerfile"
    content: |
      FROM ubuntu:24.04
      ARG DEBIAN_FRONTEND=noninteractive
      # Kopiujemy nasz artefakt z maszyny docelowej do wnętrza
kontenera
      COPY {{ artifact_name }} /tmp/
      # Instalujemy Neovima rozwiązując jego zależności (apt-get)
      RUN apt-get update && apt-get install -y /tmp/{{ artifact_name }}
&& rm -rf /var/lib/apt/lists/*
      WORKDIR /workspace

- name: Zbuduj obraz kontenera z wdrożoną aplikacją
  ansible.builtin.command:
    cmd: docker build -t neovim-deployed-app .
    chdir: "{{ deploy_dir }}"

# -----
# Smoke test paczki z opcją --headless
# -----
- name: Uruchom aplikację w kontenerze (Zwrócenie wersji Neovima)
  ansible.builtin.command:
    cmd: docker run --rm neovim-deployed-app nvim --headless -v
  register: app_verification_output

- name: Wyświetl dowód działania (Weryfikacja)
  ansible.builtin.debug:
    msg: "Aplikacja działa poprawnie! Zwrócona wersja: {{
app_verification_output.stdout_lines[0] }}"

# -----
# Czyszczenie środowiska docelowego
# -----
- name: Usuń obraz Dockera (sprzątanie)
  ansible.builtin.command:
    cmd: docker rmi neovim-deployed-app
  ignore_errors: yes

- name: Usuń katalog wdrożeniowy i pliki binarne z maszyny docelowej
  ansible.builtin.file:
    path: "{{ deploy_dir }}"
    state: absent

```

```

bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab8$ ansible-playbook -i inventory.ini neovim_deploy.yml
    "changed": false,
    "msg": "Sanity check zakończony sukcesem."
  }

TASK [Zainstaluj Dockera (upewnij się wprost, że istnieje)] *****
ok: [ansible-target]

TASK [Upewnij się, że usługa Docker jest uruchomiona] *****
ok: [ansible-target]

TASK [Utwórz katalog wdrożeniowy na maszynie docelowej] *****
ok: [ansible-target]

TASK [Skopiuj paczkę DEB z Orchestratora na maszynę docelową] *****
ok: [ansible-target]

TASK [Utwórz Dockerfile, by umieścić/udostępnić plik w kontenerze] *****
ok: [ansible-target]

TASK [Zbuduj obraz kontenera z wdrożoną aplikacją] *****
changed: [ansible-target]

TASK [Uruchom aplikację w kontenerze (Zwrócenie wersji Neovima)] *****
changed: [ansible-target]

TASK [Wyświetl dowód działania (Weryfikacja)] *****
ok: [ansible-target] => {
  "msg": "Aplikacja działa poprawnie! Zwrócona wersja: NVIM v0.13.0-dev-249+gf2cc0a249d"
}

TASK [Usuń obraz Dockera (sprzątanie)] *****
changed: [ansible-target]

TASK [Usuń katalog wdrożeniowy i pliki binarne z maszyny docelowej] *****
changed: [ansible-target]

PLAY RECAP *****
ansible-target      : ok=12  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

Następnie owijam tego playbooka w tzw. rolę za pomocą polecenia `ansible-galaxy init role<rola>`. Tworzy się struktura plików, w której muszę uzupełnić odpowiednie pliki. Playbook przeniesiony zostaje do `neovim_deploy/tasks/main.yml`, dodatkowo podajemy potrzebne metadane w pliku `neovim_deploy/main.yml`, takie jak nazwa autora, organizacji, wymagania systemowe, licencje, tagi itp. :

```

galaxy_info:
  author: BB419678
  description: your role description
  company: your company (optional)

# If the issue tracker for your role is not on github, uncomment the
# next line and provide a value
# issue_tracker_url: http://example.com/issue/tracker

# Choose a valid license ID from https://spdx.org - some suggested
licenses:
  # - BSD-3-Clause (default)
  # - MIT
  # - GPL-2.0-or-later
  # - GPL-3.0-only
  # - Apache-2.0
  # - CC-BY-4.0
  license: MIT

```

```
min_ansible_version: 2.1

platforms:
- name: Ubuntu
  versions:
    - "24.04"

# If this a Container Enabled role, provide the minimum Ansible Container
version.
# min_ansible_container_version:

#
# Provide a list of supported platforms, and for each platform a list of
versions.
# If you don't wish to enumerate all versions for a particular platform,
use 'all'.
# To view available platforms and versions (or releases), visit:
# https://galaxy.ansible.com/api/v1/platforms/
#
# platforms:
# - name: Fedora
#   versions:
#     - all
#     - 25
# - name: SomePlatform
#   versions:
#     - all
#     - 1.0
#     - 7
#     - 99.99

galaxy_tags:
- neovim
- docker
- deploy
- testing

# List tags for your role here, one per line. A tag is a keyword that
describes
# and categorizes the role. Users find roles by searching for tags. Be
sure to
# remove the '[]' above, if you add tags to this list.
#
# NOTE: A tag is limited to a single word comprised of alphanumeric
characters.
#           Maximum 20 tags per role.

dependencies: []
# List your role dependencies here, one per line. Be sure to remove the
'[]' above,
```

```
# if you add dependencies to this list.
```

Wnioski

Narzędzia takie jak Ansible pozwalają na bezagentowe i deklaratywne zarządzanie konfiguracją wielu maszyn jednocześnie. Wykorzystanie plików inwentaryzacji (Inventory) oraz playbooków znacząco automatyzuje powtarzalne procesy operacyjne, takie jak aktualizacje pakietów czy wdrażanie aplikacji (np. paczek DEB w kontenerach). Z kolei mechanizm ról (Ansible Galaxy) pozwala na modułarną organizację kodu, ułatwiając jego wielokrotne użycie.

Temat 9

Cel zajęć - Przygotowanie źródła instalacyjnego dla maszyny wirtualnej Fedora

W ramach zajęć musiałem utworzyć źródło instalacji nienadzorowanej dla systemu operacyjnego hostującego oraz przeprowadzić instalację systemu, który po uruchomieniu rozpocznie hostowanie naszego programu.

Na początku pobrałem obraz fedora 44 server z wbudowanymi pakietami, który przyjmuje pliki odpowiedzi w menu grub zgodnie z instrukcją, następnie utworzyłem prostą maszynę wirtualną z tym obrazem:



Następnie znalazłem plik `/root/anaconda-ks.cfg`, który wygląda następująco:

```
# Generated by Anaconda 44.30
# Keyboard layouts
keyboard --vckeymap=pl --xlayouts='pl'
# System language
lang pl_PL.UTF-8

%packages
@^custom-environment

%end

# Run the Setup Agent on first boot
firstboot --enable

# Generated using Blivet version 3.13.2
ignoredisk --only-use=sda
# System bootloader configuration
bootloader --location=mbr --boot-drive=sda
```



```
autopart
# Partition clearing information
clearpart --none --initlabel

# System timezone
timezone Europe/Warsaw --utc

# Root password
rootpw --iscrypted --allow-ssh
$y$j9T$6CsVoonZjNdD2MGrm4x4W2sC$UqWmSiLEoK3lJCSE6ijHQ/Lv8B7D4eWQGfKXv5sEA78
```

Mamy tutaj podstawowe parametry do konfiguracji systemu, takie jak kodowanie, język, paczki, rodzaj partycjonowania pamięci, hash hasła użytkownika root itp.

Na podstawie tego pliku tworzę własny plik konfiguracyjny, który wystawiam na maszynie w postaci servera http za pomocą polecenia `python3 http.server`:

```
# === główna konfiguracja ===
text
lang pl_PL.UTF-8
keyboard pl
timezone Europe/Warsaw

# Źródło instalacji
url --mirrorlist=http://mirrors.fedoraproject.org/mirrorlist?repo=fedora-44&arch=x86_64
repo --name=update --
mirrorlist=http://mirrors.fedoraproject.org/mirrorlist?repo=updates-released-f44&arch=x86_64

# Formatowanie w kółko i automatyczne partycje
clearpart --all --initlabel
autopart

# Nazwa hosta i użytkownicy (inne niż domyślne)
network --bootproto=dhcp --hostname=fedora-devops-host
rootpw --plaintext haslo123
user --name=fedora --password=haslo123 --groups=wheel

# restart po stworzeniu użytkownika
reboot

# Wybór pakietów do instalacji
%packages
@core
docker
wget
%end

# === (%post) ===
```

```
%post
# %post wypisze nam wszystkie informacje na ekran
exec < /dev/tty3 > /dev/tty3 2>&1
chvt 3
echo "=== konfiguracja po instalacji danych ==="

# docker po reboocie
systemctl enable docker

# Rozwiązanie problemu: Docker nie działa w instalatorze.
# Tworzymy skrypt "First Boot", który wykona się przy pierwszym
uruchomieniu
cat << 'EOF' > /usr/local/bin/deploy-neovim.sh
#!/bin/bash
sleep 15 # Czekamy na sieć i Dockera

# pobieramy artefakt z naszej maszyny ubuntu po adresie ip!
wget http://172.25.32.137:8000/neovim-final.deb -O /tmp/neovim-final.deb

# Tworzymy obraz Dockera
cat << 'DOCKER' > /tmp/Dockerfile
FROM ubuntu:24.04
ARG DEBIAN_FRONTEND=noninteractive
COPY neovim-final.deb /tmp/
RUN apt-get update && apt-get install -y /tmp/neovim-final.deb && rm -rf
/var/lib/apt/lists/*
CMD ["nvim", "--headless", "-v"]
DOCKER

cd /tmp
docker build -t neovim-app .
# test czy działa
docker run --rm neovim-app > /root/neovim-działa.txt
EOF

chmod +x /usr/local/bin/deploy-neovim.sh

# Rejestrujemy skrypt jako jednorazową usługę startową
cat << 'EOF' > /etc/systemd/system/neovim-deploy.service
[Unit]
Description=Deploy Neovim Container on First Boot
After=network-online.target docker.service
Requires=docker.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/deploy-neovim.sh
RemainAfterExit=yes

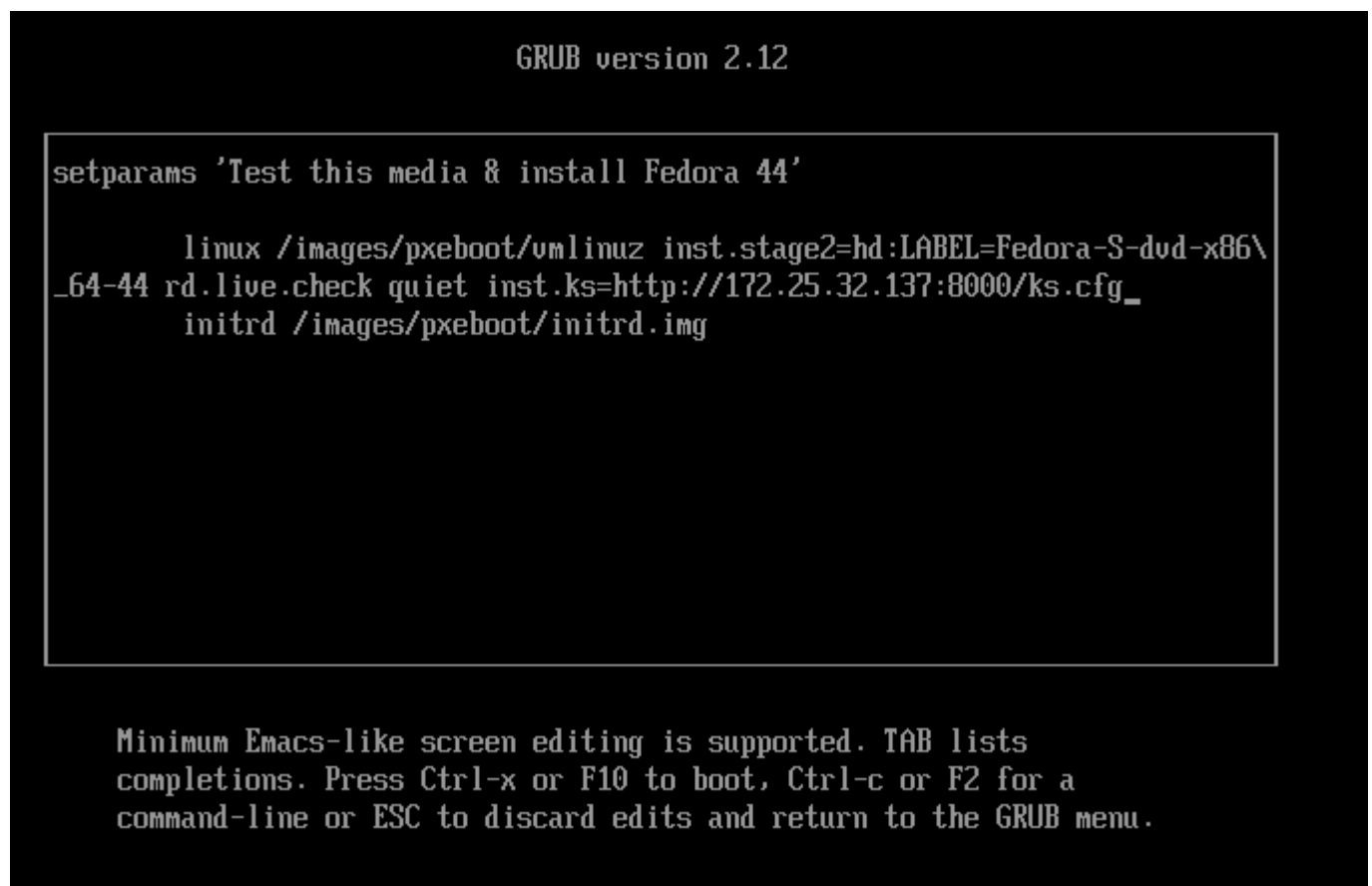
[Install]
WantedBy=multi-user.target
EOF

systemctl enable neovim-deploy.service
```

```
echo "=== koniec konfiguracji ==="  
chvt 1  
%end
```

Ustawiamy strefę czasową systemu na Warszawę, konfigurujemy patrycjonowanie, ściągamy wszystkie potrzebne zależności z zdefiniowanych źródeł instalacji, dodatkowo przekierowujemy wszystkie do stdout, żeby widzieć wyniki wszystkich poleceń. Następnie pobieramy nasz artefakt z pipeline'u, tworzymy oddzielny obraz dockera, na którym włączamy oprogramowanie w celu sprawdzenia jego działania. Na koniec zapisujemy wyniki w pliku tekstowym, usuwamy obraz i pliki wykonawcze i sprzątamy system.

Podczas instalacji ISO na kolejnej maszynie fedora trzeba ustawić parametr ints.ks na adres naszego serwera w pythonie, tak aby system pobrał ten plik i skonfigurował się zgodnie z naszymi oczekiwaniami:



Po ekranie GRUB system przystępuje do instalacji z podanym wcześniej plikiem konfiguracji.

```

=====
=====
Installation

1) [x] Ustawienia języka                2) [x] Ustawienia czasu
    (Polish (Poland))                    (Strefa czasowa Europe/Warsaw)
3) [x] Źródło instalacji                4) [x] Wybór oprogramowania
    (http://mirrors.fedoraproject.or    (Wybrano oprogramowanie
    g/mirrorlist?repo=fedora-          niestandardowe)
    44&arch=x86_64)
5) [x] Miejsce docelowe instalacji      6) [x] Konfiguracja sieci
    (Wybrano automatyczne              (Połączono: eth0)
    partycjonowanie)

=====
=====
Progress

Ustawianie środowiska instalacji
.
Konfigurowanie urządzeń do przechowywania danych
Tworzenie disklabel na /dev/sda
Tworzenie xfs na /dev/sda2
Tworzenie lvm pv na /dev/sda3
Tworzenie xfs na /dev/mapper/fedora_fedora--devops--host-root
Tworzenie efi na /dev/sda1
.
Wykonywanie skryptów przedinstalacyjnych
.
Wykonywanie zadań przedinstalacyjnych
.
Instalowanie oprogramowania
Pobieranie pakietów
Pobieranie plików RPM: 458, ukończono: 8,75 MiB/756,61 MiB (1%).
Pobieranie plików RPM: 458, ukończono: 19,17 MiB/756,61 MiB (2%).
Pobieranie plików RPM: 458, ukończono: 27,51 MiB/756,61 MiB (3%).
Pobieranie plików RPM: 458, ukończono: 40,69 MiB/756,61 MiB (5%).
Pobieranie plików RPM: 458, ukończono: 47,33 MiB/756,61 MiB (6%).
Pobieranie plików RPM: 458, ukończono: 60,08 MiB/756,61 MiB (7%).
Pobieranie plików RPM: 458, ukończono: 73,81 MiB/756,61 MiB (9%).
Pobieranie plików RPM: 458, ukończono: 89,44 MiB/756,61 MiB (11%).
Pobieranie plików RPM: 458, ukończono: 101,02 MiB/756,61 MiB (13%).
Pobieranie plików RPM: 458, ukończono: 112,38 MiB/756,61 MiB (14%).
Pobieranie plików RPM: 458, ukończono: 124,72 MiB/756,61 MiB (16%).
Pobieranie plików RPM: 458, ukończono: 135,97 MiB/756,61 MiB (17%).
Pobieranie plików RPM: 458, ukończono: 147,87 MiB/756,61 MiB (19%).

[anaconda1:main* 2:shell 3:log 4:storage-log 5:program-log 6:packaging-log
Stan: Uruchomiony

```

Jeżeli natomiast stracimy połączenie z serwerem pliku konfiguracyjnego, dostaniemy taki komunikat, który informuje nas że plik kickstart nie został znaleziony!

```
anaconda 44.30-2.fc44 for Fedora 44 started.  
* installation log files are stored in /tmp during the installation  
* shell is available on TTY2 and in second Tmux pane (ctrl+b, then press 2)  
* when reporting a bug add logs from /tmp as separate text/plain attachments  
07:06:51 Kickstart file /run/install/ks.cfg is missing.
```

Pane is dead (status 1, Fri May 22 07:06:51 2026)

[anaconda11:main* 2:shell 3:log 4:storage-log 5:program-log 6:packaging-log

Po zakończeniu poprawnej instalacji logujemy się na użytkownika fedora z hasłem haslo123 (tylko dla celów edukacyjnych, nie jest to bezpieczne!)

```
Fedora Linux 44 (Forty Four)
Kernel 7.0.9-205.fc44.x86_64 on x86_64 (tty1)

Try contacting this VM's SSH server via 'ssh vsock%4294967295' from host.

fedora-devops-host login: fedora
Password:
[fedora@fedora-devops-host ~]$_
```

Sprawdzamy, czy obraz sprawdził wersje naszego oprogramowania - należy pamiętać, żeby używać sudo w ścieżce /root na fedorze.

```
Fedora Linux 44 (Forty Four)
Kernel 7.0.9-205.fc44.x86_64 on x86_64 (tty1)

Try contacting this VM's SSH server via 'ssh vsock%4294967295' from host.

fedora-devops-host login: fedora
Password:
[fedora@fedora-devops-host ~]$_ nvim
-bash: nvim: command not found
[fedora@fedora-devops-host ~]$_ neovim'
> ^C
[fedora@fedora-devops-host ~]$_ neovim
-bash: neovim: command not found
[fedora@fedora-devops-host ~]$_ l
-bash: l: command not found
[fedora@fedora-devops-host ~]$_ ls
[fedora@fedora-devops-host ~]$_ cat /root/neovim-dziala.txt
cat: /root/neovim-dziala.txt: Brak dostępu
[fedora@fedora-devops-host ~]$_ sudo !!
sudo cat /root/neovim-dziala.txt

Mamy, że lokalny administrator udzielił odpowiedniego szkolenia.
Zwykle sprowadza się ono do tych trzech rzeczy:

    1) należy respektować prywatność innych,
    2) należy myśleć przed pisaniem,
    3) z dużą władzą wiąże się duża odpowiedzialność.

Ze względów bezpieczeństwa wpisywane hasło nie będzie widoczne.

[sudo] hasło użytkownika fedora:
NVIM v0.13.0-dev-249+gf2cc0a249d
Build type: RelWithDebInfo
LuaJIT 2.1.1774896198
Run "nvim -U1 -v" for more info
[fedora@fedora-devops-host ~]$_
```

Dodatkowo można jeszcze zautomatyzować cały proces tworzenia takiej maszyny. Dla hyperV możemy użyć skryptu, który bierze z predefiniowanych folderów obrazy, tworzy maszynę z zadaną ilością pamięci ram, miejsca na dysku itp. oraz konfiguruje zabezpieczenia. Przykładowy skrypt może wyglądać tak:

```
# Ustawienia
$VMName = "Fedora-Kickstart"
$ISOPath = "E:\_Install\Inne\linux_iso\Fedora-Server-netinst-x86_64-44-1.7.iso" # <--- zmienna sciezka
$SwitchName = "Default Switch" # <--- mozemy takze zmieniać switche do sieci

# Utworzenie nowej maszyny Generacji 2 z 2GB RAM i dyskiem 20GB
New-VM -Name $VMName -MemoryStartupBytes 2048MB -Generation 2 -NewVHDPATH "C:\ProgramData\Microsoft\Windows\Hyper-V\Virtual Hard Disks\$VMName.vhdx" -NewVHDSIZEBytes 20GB -SwitchName $SwitchName

# Skonfigurowanie Secure Boot pod Linuksa
Set-VMFirmware -VMName $VMName -SecureBootTemplate "MicrosoftUEFICertificateAuthority"

# ISO fedory do instalacji
Add-VMDvdDrive -VMName $VMName -Path $ISOPath

# Kolejność bootowania systemu
$DVDDrive = Get-VMDvdDrive -VMName $VMName
Set-VMFirmware -VMName $VMName -FirstBootDevice $DVDDrive

# Uruchomienie maszyny
Start-VM -VMName $VMName
```

Skrypt ten można włączyć w środowisku Powershell jako administrator - wynikiem jest maszyna wirtualna o takiej samej specyfikacji jak w skrypcie:

 fedora-cfg-install	Wyłączony				12.0
 Fedora-Kickstart	Uruchomiony	8%	2048 MB	00:00:16	12.0

```
Administrator: Windows PowerShell
PS E:\Moje Dokumenty\ITE\Semestr_6\Metody_DevOps\Laboratoria> .\lab9.ps1

Name                State CPUUsage(%) MemoryAssigned(M) Uptime   Status           Version
-----
Fedora-Kickstart Off    0           0                00:00:00 Działa normalnie 12.0

PS E:\Moje Dokumenty\ITE\Semestr_6\Metody_DevOps\Laboratoria>
```

Wnioski

Użycie plików odpowiedzi (takich jak ks.cfg) umożliwia całkowicie zautomatyzowaną, nienadzorowaną instalację systemu operacyjnego, co drastycznie skraca czas wdrażania nowych węzłów. Kluczowym wyzwaniem jest fakt, że podczas fazy instalacyjnej (w sekcji %post) usługi takie jak Docker nie są jeszcze uruchomione. Wymusza to stosowanie mechanizmów opóźnionych, takich jak skrypty "First Boot" wdrażane jako usługi systemd.

Temat 10

Cel zajęć - Wdrażanie oprogramowania na zarządzalne kontenery za pomocą oprogramowania Kubernetes

W ramach ćwiczenia wykorzystałem obraz `nginx:alpine` zamiast wcześniej zbudowanego oprogramowania, gdyż pozwoli to na lepsze zobrazowanie różnych scenariuszy zmiany konfiguracji wdrożeń nowszych wersji.

Na początku instaluje minikube oraz kubectl zgodnie z dokumentacją:

```
curl -LO
https://github.com/kubernetes/minikube/releases/latest/download/minikube-
linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-
linux-amd64
```

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Włączam klaster na 2 CPU i 2GB RAM:

```
minikube start --cpus=2 --memory=2048 --driver=docker
```

Sprawdzam dashboard (nawiasem mówiąc jest pusty):

```
minikube dashboard &
```

Dodatkowo tworzę prosty dockerfile dla mojej aplikacji, który zostanie później użyty do deploymentu jej różnych wersji:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Na podstawie tych faktów przystępuje do czynów. Na początku ładuję wcześniej utworzony obraz do minikube, "modyfikuję" wersję programu, buduję wersję nr. 2, ładuję ją do minikube, dodatkowo tworzę trzecią "zepsutą" wersję aplikacji i ponownie ładuję ją do klastra.


```

• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ minikube image load moj-app:v1
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ echo "<h1>Wersja V2 - po aktualizacji!</h1>" > index.html
bash: !: event not found
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ echo "<h1>Wersja V2 - po aktualizacji</h1>" > index.html
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ docker build -t moj-app:v2 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.584kB
Step 1/2 : FROM nginx:alpine
--> da954fb959a3
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
--> e7acdfc8b4c5
Successfully built e7acdfc8b4c5
Successfully tagged moj-app:v2
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ minikube image load moj-app:v2
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ cat << 'EOF' > Dockerfile.bad
FROM nginx:alpine
CMD ["komenda-ktora-nie-istnieje"]
EOF
docker build -t moj-app:v3-broken -f Dockerfile.bad .
minikube image load moj-app:v3-broken
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.608kB
Step 1/2 : FROM nginx:alpine
--> da954fb959a3
Step 2/2 : CMD ["komenda-ktora-nie-istnieje"]
--> Running in 8903737944fb
--> Removed intermediate container 8903737944fb
--> 0861caae6c8c
Successfully built 0861caae6c8c
Successfully tagged moj-app:v3-broken

```

Na początku sprawdzam działanie pojedynczego poda, gdzie przekierowuje nginx na port 8081 i sprawdzam działanie narzędziem curl:

```

kubectrl run moj-pod --image=moj-app:v1 --port=80 --image-pull-policy=Never
--labels app=moj-pod

kubectrl port-forward pod/moj-pod 8081:80 &

```

```

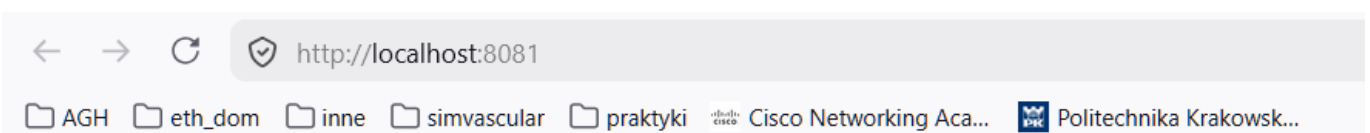
• bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/1ab10$ Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
Handling connection for 8081

```

```

• bartosz123@ansible-master:~/MDO2026s_ITE$ curl localhost:8081
• <h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>bartosz123@ansible-master:~/MDO2026s_ITE$

```



To jest wersja V1 aplikacji na laboratoria kubernetes!

Jak widać, pokazuje się wersja 1 aplikacji narazie, co jest zgodne z poleceniem.

Pod można usunąć poleceniem:

```
kubectrl delete pod moj-pod
```

Następnie tworze plik deployment.yaml, dzięki któremu moge zautomatyzować wdrożenie wersji aplikacji na klaster:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-deployment
spec:
  replicas: 4
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: kontener-aplikacji
          image: moj-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Tworzymy tutaj deployment, który utworzy 4 pody zawierają wersję v1 nginx. Dodatkowo dodaję jeszcze serwis, który będzie przekazywał port jednego z podów na local, dzięki czemu będzie można sprawdzić działanie deploymentu.

Włączam deployment poniższą komendą:

```
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment created
service/lab-service created
```

Sprawdzam status deploymentu - czy nie ma żadnych błędów (nie ma):

```
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

Dodatkowo na dashboardzie widać nasze 4 pody z deploymentu, każdy z nich używa obrazu moj-app:v1, co jest zgodne z plikiem wdrożenia.

Workload Status

Running 1

Deployments

Running 1

Pods

Running 1

Replica Sets

Deployments

Name	Images	Labels	Pods	Created ↑
lab-deployment	moj-app:v1	-	4 / 4	44 seconds ago

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑
lab-deployment-7864cfc699-5gpbz	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	44 seconds ago
lab-deployment-7864cfc699-791sp	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	44 seconds ago
lab-deployment-7864cfc699-gbfz7	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	44 seconds ago
lab-deployment-7864cfc699-npcwn	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	44 seconds ago
moj-pod	moj-app:v1	app: moj-pod	minikube	Running	0	-	-	13 minutes ago

Replica Sets

Name	Images	Labels	Pods	Created ↑
lab-deployment-7864cfc699	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	4 / 4	44 seconds ago

Przekazuje porty i sprawdzam działanie deploymentu - jak widać działa poprawnie na porcie 8082.

```

bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl port-forward svc/lab-service 8082:80 &
[3] 40718
bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab10$ Forwarding from 127.0.0.1:8082 -> 80
Forwarding from [::1]:8082 -> 80
Handling connection for 8082
bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab10$

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 2
bash - MDO2026s_ITE

bartosz123@ansible-master:~/MDO2026s_ITE$ minikube start --cpus=2 --memory=2048 --driver=docker
🌟 Enabled addons: default-storageclass, storage-provisioner, dashboard
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
bartosz123@ansible-master:~/MDO2026s_ITE$ minikube dashboard &
[1] 29040
bartosz123@ansible-master:~/MDO2026s_ITE$ 🤖 Verifying dashboard health ...
🚀 Launching proxy ...
🔍 Verifying proxy health ...
🌐 Opening http://127.0.0.1:39461/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
bartosz123@ansible-master:~/MDO2026s_ITE$ curl localhost:8081
bartosz123@ansible-master:~/MDO2026s_ITE$
bartosz123@ansible-master:~/MDO2026s_ITE$ kubectl apply -f deployment.yaml
error: the path "deployment.yaml" does not exist
bartosz123@ansible-master:~/MDO2026s_ITE$ curl localhost:8082
<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
bartosz123@ansible-master:~/MDO2026s_ITE$

```

Następnie zmieniam liczbę podów w wdrożeniu z 4 na 8 za pomocą polecenia `kubectl scale` i ponownie sprawdzam ich liczbę. Analogicznie można skalować w dół oraz do 0, daje to podobne wyniki do tego:

```

bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl scale deployment/lab-deployment --replicas=8
kubectl get pods # 8 kontenerów
kubectl scale deployment/lab-deployment --replicas=1
kubectl scale deployment/lab-deployment --replicas=0
kubectl scale deployment/lab-deployment --replicas=4
deployment.apps/lab-deployment scaled
NAME                                READY    STATUS             RESTARTS    AGE
lab-deployment-7864cfc699-4wmfk      0/1      ContainerCreating   0            0s
lab-deployment-7864cfc699-5gpbz      1/1      Running             0            4m46s
lab-deployment-7864cfc699-791sp      1/1      Running             0            4m46s
lab-deployment-7864cfc699-b97cf      0/1      ContainerCreating   0            0s
lab-deployment-7864cfc699-fh9sv      0/1      ContainerCreating   0            0s
lab-deployment-7864cfc699-gbfz7      1/1      Running             0            4m46s
lab-deployment-7864cfc699-jnfzd      0/1      Pending             0            0s
lab-deployment-7864cfc699-npcwn      1/1      Running             0            4m46s
moj-pod                             1/1      Running             0            17m
deployment.apps/lab-deployment scaled
deployment.apps/lab-deployment scaled
deployment.apps/lab-deployment scaled
bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab10$

```

Aktualizuję obraz do wersji moj-app:v2 za pomocą polecenia `kubectl set image` oraz sprawdzam stan klastra poleceniem `kubectl rollout status`

```
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ kubectl set image deployment/lab-deployment kontener-aplikac
ji=moj-app:v2
deployment.apps/lab-deployment image updated
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$
```

Ponownie przekierowuje porty i sprawdzam działanie za pomocą narzędzia curl - wynikiem jest komunikat "Wersja V2 - po aktualizacji" - pody poprawnie zmieniły obraz w ramach modyfikacji deploymentu.

```
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ killall kubectl
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ kubectl get pods
[1]- Terminated          kubectl port-forward pod/moj-pod 7070:80
[2]+ Terminated          kubectl port-forward pod/moj-pod 8081:80
NAME                      READY   STATUS    RESTARTS   AGE
lab-deployment-7d9dfd9744-g8h8x  1/1     Running   0           3m5s
lab-deployment-7d9dfd9744-njkpx  1/1     Running   0           3m7s
lab-deployment-7d9dfd9744-q5vqh  1/1     Running   0           3m7s
lab-deployment-7d9dfd9744-zkhcz  1/1     Running   0           3m6s
moj-pod                      1/1     Running   0           21m
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ kubectl port-forward svc/lab-service 8082:80 &
[1] 45413
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab10$ Forwarding from 127.0.0.1:8082 -> 80
Forwarding from [::1]:8082 -> 80
Handling connection for 8082
```

The screenshot shows a terminal window with the following commands and output:

```
bartosz123@ansible-master:~/MD02026s ITE$ minikube dashboard &
[1] 45760
bartosz123@ansible-master:~/MD02026s ITE$ Verifying dashboard health ...
Launching proxy ...
Verifying proxy health ...
Opening http://127.0.0.1:35293/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
in your default browser...
bartosz123@ansible-master:~/MD02026s ITE$
```

On the right side of the terminal, the following command and output are shown:

```
bartosz123@ansible-master:~/MD02026s ITE$ curl localhost:8082
<h1>Wersja V2 - po aktualizacji</h1>
bartosz123@ansible-master:~/MD02026s ITE$
```

Dodatkowo na dashboardzie wszystkie pody początkowego deploymentu teraz mają obraz V2, natomiast widzimy dwa deploymenty w ReplicaSet, jeden V1, drugi V2.

The screenshot shows the Kubernetes dashboard with the following sections:

- Deployments:** A table showing the deployment status.
- Pods:** A table showing the pod status.
- Replica Sets:** A table showing the replica set status.

Name	Images	Labels	Pods	Created
lab-deployment	moj-app:v2		4 / 4	11 minutes ago

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created
lab-deployment-7d9dfd9744-g8h8x	moj-app:v2	app: web-app, pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	6 minutes ago
lab-deployment-7d9dfd9744-njkpx	moj-app:v2	app: web-app, pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	6 minutes ago
lab-deployment-7d9dfd9744-q5vqh	moj-app:v2	app: web-app, pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	6 minutes ago
lab-deployment-7d9dfd9744-zkhcz	moj-app:v2	app: web-app, pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	6 minutes ago
moj-pod	moj-app:v1	app: moj-pod	minikube	Running	0	-	-	24 minutes ago

Name	Images	Labels	Pods	Created
lab-deployment-7d9dfd9744	moj-app:v2	app: web-app, pod-template-hash: 7d9dfd9744	4 / 4	6 minutes ago
lab-deployment-7884cfd999	moj-app:v1	app: web-app, pod-template-hash: 7884cfd999	0 / 0	11 minutes ago

Następnie zmieniam wersję obrazu na V3 w celu sprawdzenia czy klaster przyjmie wadliwy obraz - wynikiem jest zepsucie połowy kontenerów, w związku z czym trzeba zrobić `kubectl rollout undo` aby powrócić do poprzedniego stanu.

```
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl set image deployment/lab-deployment kontener-aplikac
ji=moj-app:v3-broken
deployment.apps/lab-deployment image updated
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab-deployment-57cd55bb47-n7wdm     0/1     Error     1 (7s ago)  7s
lab-deployment-57cd55bb47-wv5qj     0/1     Error     1 (7s ago)  7s
lab-deployment-7d9dfd9744-njkpx     1/1     Running   0           10m
lab-deployment-7d9dfd9744-q5vqh     1/1     Running   0           10m
lab-deployment-7d9dfd9744-zkhcz     1/1     Running   0           10m
moj-pod                             1/1     Running   0           28m
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl rollout history deployment/lab-deployment
deployment.apps/lab-deployment
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
3          <none>
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$ kubectl rollout undo deployment/lab-deployment
Warning: resource deployments/lab-deployment was previously managed with 'kubectl apply'. Rolling back will not update the kub
ectl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' operat
ions. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/lab-deployment rolled back
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab10$
```

Następnie tworzę prosty skrypt, który sprawdza status deploymentu i zwraca odpowiedni komunikat:

```
$ check_deploy.sh U X
ITE > GCL1 > BB419678 > lab10 > $ check_deploy.sh
1  #!/bin/bash
2  # sprawdzamy przez 60 sekund
3  kubectl rollout status deployment/lab-deployment --timeout=60s
4
5  if [ $? -eq 0 ]; then
6      echo "SUKCES: Wdrozenie udane i zdazylo wstac w 60s!"
7  else
8      echo "PORAZKA: Wdrozenie zawislo lub napotkalo problem (np. CrashLoopBackOff)."
```

Przechodzę do kwestii rodzajów wdrożeń - zacznę od recreate. Ten typ wdrożenia zabije wszystkie pody ze starą wersją, po czym podnosi je z nowszą wersją oprogramowania:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-deployment
spec:
  replicas: 4
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
```

```
    app: web-app
spec:
  containers:
    - name: kontener-aplikacji
      image: moj-app:v1
      imagePullPolicy: Never
      ports:
        - containerPort: 80
  ---
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

	lab-deployment-7864cfc699-bt5gf	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	23 seconds ago	
	lab-deployment-7864cfc699-nldf5	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	23 seconds ago	
	lab-deployment-7864cfc699-qqz5d	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	23 seconds ago	
	lab-deployment-7864cfc699-t74ng	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	minikube	Running	0	-	-	23 seconds ago	
	Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑	
	lab-deployment-7d9dfd9744-	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	18 seconds ago	
	lab-deployment-7d9dfd9744-mjxb4	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	18 seconds ago	
	lab-deployment-7d9dfd9744-mkfkx	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	18 seconds ago	
	lab-deployment-7d9dfd9744-zxx4n	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	minikube	Running	0	-	-	18 seconds ago	

Kolejny jest rolling update, gdzie określona część kontenerów zostanie podmieniona w płynny sposób, a reszta pozostanie w tym czasie w wersji poprzedniej, aby uniknąć przerwy w dostarczaniu usług.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-deployment
spec:
```

```
replicas: 4
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 25% # 1 kontener przy 4 replikach
selector:
  matchLabels:
    app: web-app
template:
  metadata:
    labels:
      app: web-app
  spec:
    containers:
      - name: kontener-aplikacji
        image: moj-app:v2
        imagePullPolicy: Never
        ports:
          - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Wynikiem tego jest 3 kontenery w wersji v2 oraz 1 w wersji v1, który pod koniec zmienia się w kontener wersji v2.

Pods				
	Name	Images	Labels	n
●	lab-deployment-7d9dfd974btl6r	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	n
●	lab-deployment-7d9dfd974vgf88	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	n
●	lab-deployment-7d9dfd974...	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	n
●	lab-deployment-7d9dfd974t6jjb	moj-app:v2	app: web-app pod-template-hash: 7d9dfd9744	
●	lab-deployment-7864cfc69rw24q	moj-app:v1	app: web-app pod-template-hash: 7864cfc699	
●	moj-pod	moj-app:v1	app: moj-pod	n

Na koniec mamy deployment typu canary, gdzie najpierw musimy wyłączyć wszystkie pody podchodzący pod nasz pierwotny deployment. Ten typ wdrożeń polega na tagach, gdzie tylko ustalona część klientów ma dostęp do podów z nowszą wersją oprogramowania, w porównaniu do Rolling Update to przejście na nowszą wersję jest prawie natychmiastowe, oczywiście po poprawnie zaliczonym zbiorze testów.

Do zrobienia wdrożenia ponownie modyfikujemy nasz plik wdrożenia:

```
---
# (Wspólny punkt wejścia)
apiVersion: v1
kind: Service
metadata:
  name: canary-service
spec:
```

```
selector:
  app: aplikacja-lab # Złapie wszystkie pody z tą etykietą
ports:
  - protocol: TCP
    port: 80
    targetPort: 80

---
# wdrozenie stabilne v1 - 75% ruchu
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-stable
spec:
  replicas: 3
  selector:
    matchLabels:
      app: aplikacja-lab
      track: stable # Unikalna etykieta stabilna
  template:
    metadata:
      labels:
        app: aplikacja-lab
        track: stable
    spec:
      containers:
        - name: kontener
          image: moj-app:v1
          imagePullPolicy: Never
          ports:
            - containerPort: 80

---
# wdrozenie "canary" - 25% ruchu
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-canary
spec:
  replicas: 1
  selector:
    matchLabels:
      app: aplikacja-lab
      track: canary # Unikalna etykieta kanarka
  template:
    metadata:
      labels:
        app: aplikacja-lab
        track: canary
    spec:
      containers:
        - name: kontener
          image: moj-app:v2
          imagePullPolicy: Never
```

```
ports:
- containerPort: 80
```

```
20 spec:
21   containers:
22     - name: kontener-aplikacji
23       image: moj-app:v2
24       imagePullPolicy: Never
25       ports:
26         - containerPort: 80
27
28   apiVersion: v1
29   kind: Service
30   metadata:
31     name: lab-service
32   spec:
33     selector:
34       app: web-app
35     ports:
36       - protocol: TCP
37         port: 80
38         targetPort: 80
```

```
bartosz123@ansible-master: ~/MDO2026s ITE/ITE/GCL1/8B419678/1ab10
$ kubectl apply -f rolling_updates.yaml
deployment.apps/lab-deployment configured
service/lab-service unchanged
bartosz123@ansible-master: ~/MDO2026s ITE/ITE/GCL1/8B419678/1ab10
$ kubectl apply -f recreate.yaml
deployment.apps/lab-deployment configured
service/lab-service unchanged
bartosz123@ansible-master: ~/MDO2026s ITE/ITE/GCL1/8B419678/1ab10
$ kubectl apply -f rolling_updates.yaml
deployment.apps/lab-deployment configured
service/lab-service unchanged
bartosz123@ansible-master: ~/MDO2026s ITE/ITE/GCL1/8B419678/1ab10
$ kubectl delete deployment lab-deployment
deployment.apps "lab-deployment" deleted from default namespace
bartosz123@ansible-master: ~/MDO2026s ITE/ITE/GCL1/8B419678/1ab10
$ kubectl delete service lab-service
service "lab-service" deleted from default namespace
```

kubernetes

default

Search

Workloads

Workload Status

Running: 1

Pods

moj-pod

moj-app:v1

app: moj-pod

Running 2

Deployments

Running 3

Pods

Running 3

Replica Sets

Name	Images	Labels	Pods	Created
app-canary	moj-app:v2		1 / 1	19 seconds ago
app-stable	moj-app:v1		3 / 3	19 seconds ago

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created
app-canary-6d75d594-chnak	moj-app:v2	app: aplikacja-lab track: canary	pod-template-hash: 6d75d594	minikube	Running	0	-	19 seconds ago
app-stable-688d773bc3-2vnp2	moj-app:v1	app: aplikacja-lab track: stable	pod-template-hash: 688d773bc3	minikube	Running	0	-	19 seconds ago
app-stable-688d773bc3-6bq9p	moj-app:v1	app: aplikacja-lab track: stable	pod-template-hash: 688d773bc3	minikube	Running	0	-	19 seconds ago
app-stable-688d773bc3-4gfc5	moj-app:v1	app: aplikacja-lab track: stable	pod-template-hash: 688d773bc3	minikube	Running	0	-	19 seconds ago
moj-pod	moj-app:v1	app: moj-pod	minikube	Running	1	-	-	4 days ago

Name	Images	Labels	Pods	Created
app-canary-6d75d594	moj-app:v2	app: aplikacja-lab pod-template-hash: 6d75d594 track: canary	1 / 1	19 seconds ago
app-stable-688d773bc3	moj-app:v1	app: aplikacja-lab pod-template-hash: 688d773bc3 track: stable	3 / 3	19 seconds ago

Dla wersji "stable" przekazujemy 75% ruchu użytkowników, natomiast reszta idzie na nasze pody z tagiem "canary". Dodatkowo sprawdzam za pomocą testowego poda czy rozkład zapytań i odpowiedzi zgadza się z plikiem konfiguracyjnym:

```
kubectl run test-pod --rm -i --tty --image=curlimages/curl -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
```

```
$ for i in $(seq 1 10); do curl -s http://canary-service; echo ""; sleep 0.5; done

<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>Wersja V2 - po aktualizacji</h1>

<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>Wersja V2 - po aktualizacji</h1>

<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>
<h1>Wersja V2 - po aktualizacji</h1>

<h1>To jest wersja V1 aplikacji na laboratoria kubernetes!</h1>

$
```

Widać, że mamy tutaj właśnie interakcje z podami "stable" oraz "canary". Stosunek poszczególnych odpowiedzi zależy pewnie od load balancera i może być niedeterministyczny.

Wnioski

Obiekty typu Deployment abstrakcyjnie zarządzają podami, dbając o utrzymanie pożądanego stanu klastra (np. zdefiniowanej liczby replik). Kubernetes natywnie wspiera bezpieczne aktualizacje wersji aplikacji, a wbudowany mechanizm historii (rollout history i undo) pozwala na błyskawiczne wycofanie zmian w przypadku wdrożenia wadliwego obrazu. Różne strategie (Recreate, Rolling Update, Canary) pozwalają dostosować proces wdrożenia do wymagań biznesowych, balansując między brakiem dostępności a płynnym przenoszeniem ruchu.

Temat 11

Kubernetes - ciąg dalszy

Na podstawie wcześniej utworzonego klastra z aplikacją nginx próbuje wyeksponować na różne sposoby. Na początku mam ustawioną liczbę podów na 12, aby nie zapchać całego RAM'u maszyny wirtualnej. Dodatkowo modyfikuje plik deployment'u - dodaje wyświetlanie identyfikatora każdego poda przy zapytaniu http:

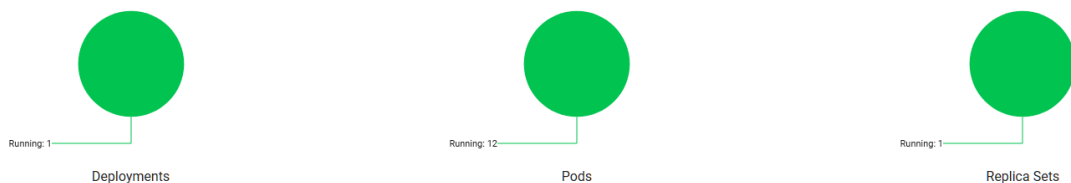
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 12 # Ogromna liczba podów!
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
```

```

labels:
  app: web-server
spec:
  containers:
  - name: nginx-container
    image: nginx:alpine
    # Pod wpisuje swoją nazwę na stronę WWW
    command: ["/bin/sh", "-c"]
    args: ["echo \"<h1>Odpowiada Pod: $HOSTNAME</h1>\" >
/usr/share/nginx/html/index.html && nginx -g 'daemon off;'" ]
    ports:
    - containerPort: 80

```

Workload Status



Deployments

Name	Images	Labels	Pods	Created ↑
web-deployment	nginx:alpine	-	12 / 12	28 minutes ago

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑
web-deployment-787c5b74bc-69tpw	nginx:alpine	app: web-server pod-template-hash: 787c5b74bc	minikube	Running	0	-	-	28 minutes ago
web-deployment-787c5b74bc-...		app: web-server						

Najpierw przekierowuje porty z jednego poda:

```

bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/1ab11$ minikube image load nginx:alpine
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/1ab11$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-deployment-787c5b74bc-69tpw     1/1     Running   0           32m
web-deployment-787c5b74bc-dwgw4     1/1     Running   0           32m
web-deployment-787c5b74bc-dx67g     1/1     Running   0           32m
web-deployment-787c5b74bc-h4xqt     1/1     Running   0           32m
web-deployment-787c5b74bc-lvbnn     1/1     Running   0           32m
web-deployment-787c5b74bc-p5pzs     1/1     Running   0           32m
web-deployment-787c5b74bc-q2zpf     1/1     Running   0           32m
web-deployment-787c5b74bc-q7grp     1/1     Running   0           32m
web-deployment-787c5b74bc-rtjbx     1/1     Running   0           32m
web-deployment-787c5b74bc-sbk7r     1/1     Running   0           32m
web-deployment-787c5b74bc-spw5      1/1     Running   0           32m
web-deployment-787c5b74bc-wrrh2     1/1     Running   0           32m
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/1ab11$ kubectl port-forward pod/web-deployment-787c5b74bc-wrrh2 8080:80
Unable to listen on port 8080: Listeners failed to create with the following errors: [unable to create listener: Error listen tcp4 127.0.0.1:8080: bind: address already in use unable to create listener
: Error listen tcp6 [::]:8080: bind: address already in use]
error: unable to listen on any of the requested ports: [{8080 80}]
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/1ab11$ kubectl port-forward pod/web-deployment-787c5b74bc-wrrh2 8084:80
Forwarding from 127.0.0.1:8084 -> 80
Forwarding from [::]:8084 -> 80
Handling connection for 8084

```

Widać, że pod poprawnie odpowiada identyfikatorem nadanym poprzez deployment:

```

bartosz123@ansible-master:~/MD02026s ITE$ curl localhost:8084
<h1>Odpowiada Pod: web-deployment-787c5b74bc-wrrh2</h1>
bartosz123@ansible-master:~/MD02026s ITE$

```

Tak samo robie następnie dla całego deploymentu - to przypisze mi dokładnie jeden pod należący do klastra:

```

^Cbartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ kubectl port-forward deployment/web-deployment 8085:80
Forwarding from 127.0.0.1:8085 -> 80
Forwarding from [::1]:8085 -> 80
Handling connection for 8085
Handling connection for 8085
Handling connection for 8085
Handling connection for 8085

```

Wynikiem jest ponownie odpowiedź z identyfikatorem poda:

```

● bartosz123@ansible-master:~/MD02026s_ITE$ curl localhost:8085
<h1>Odpowiada Pod: web-deployment-787c5b74bc-h4xqt</h1>
● bartosz123@ansible-master:~/MD02026s_ITE$ curl localhost:8085
<h1>Odpowiada Pod: web-deployment-787c5b74bc-h4xqt</h1>
● bartosz123@ansible-master:~/MD02026s_ITE$ curl localhost:8085
<h1>Odpowiada Pod: web-deployment-787c5b74bc-h4xqt</h1>
○ bartosz123@ansible-master:~/MD02026s_ITE$

```

Dodatkowo jeszcze można to zrobić na 2 sposoby:

- poleceniem: `kubectl expose deployment web-deployment --name=web-service-cmd --port=80 --target-port=80`
- oddzielnym plikiem yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: web-service-yaml
spec:
  selector:
    app: web-server
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

```

Ponownie tworze tymczasowy pod żeby przetestować działanie tej zmiany:

```

bartosz123@ansible-master:~/MD02026s_ITE$ kubectl run test-pod --rm -i --
tty --image=curlimages/curl -- sh
All commands and output from this session will be recorded in container
logs, including credentials and sensitive information passed through the
command prompt.
If you don't see a command prompt, try pressing enter.
~ $ for i in $(seq 1 10); do curl -s http://web-service-yaml; done
<h1>Odpowiada Pod: web-deployment-787c5b74bc-hvt8g</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-59tdf</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-59tdf</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-sjl7x</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-59tdf</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-qwdtx</h1>

```

```
<h1>Odpowiada Pod: web-deployment-787c5b74bc-hvt8g</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-kx44x</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-sw6fv</h1>
<h1>Odpowiada Pod: web-deployment-787c5b74bc-5jtn7</h1>
```

Jak widać, zwracane są pody o różnych identyfikatorach należących do deploymentu.

Na koniec ponownie przeskalowuje klaster:

- za pomocą polecenia scale:

```
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ kubectl
scale deployment web-deployment --replicas=10
deployment.apps/web-deployment scaled
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ kubectl
get pods
```

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Running	0	6m42s
web-deployment-787c5b74bc-59tdf	1/1	Running	0	111m
web-deployment-787c5b74bc-5jtn7	1/1	Running	0	111m
web-deployment-787c5b74bc-hvt8g	1/1	Running	0	111m
web-deployment-787c5b74bc-hww2v	1/1	Running	0	111m
web-deployment-787c5b74bc-j9k2w	1/1	Running	0	111m
web-deployment-787c5b74bc-kx44x	1/1	Running	0	111m
web-deployment-787c5b74bc-qwdtx	1/1	Running	0	111m
web-deployment-787c5b74bc-sw6fv	1/1	Running	0	111m
web-deployment-787c5b74bc-thnkb	1/1	Running	0	111m
web-deployment-787c5b74bc-zn4v5	1/1	Running	0	111m

- za pomocą ponownie zmodyfikowanego pliku yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 8 # liczba podow zmieniona na 8
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
      labels:
        app: web-server
    spec:
      containers:
        - name: nginx-container
          image: nginx:alpine
          # Pod wpisuje swoją nazwę na stronę WWW
          command: ["/bin/sh", "-c"]
```

```
args: ["echo \"<h1>Odpowiada Pod: $HOSTNAME</h1>\" >
/usr/share/nginx/html/index.html && nginx -g 'daemon off;']
ports:
- containerPort: 80
```

```
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ kubectl
apply -f web-deployment.yaml
deployment.apps/web-deployment configured
bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ kubectl
get pods
```

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Running	0	8m15s
web-deployment-787c5b74bc-59tdf	1/1	Running	0	113m
web-deployment-787c5b74bc-hvt8g	1/1	Running	0	113m
web-deployment-787c5b74bc-hww2v	1/1	Running	0	113m
web-deployment-787c5b74bc-j9k2w	1/1	Running	0	113m
web-deployment-787c5b74bc-qwdtx	1/1	Running	0	113m
web-deployment-787c5b74bc-sw6fv	1/1	Running	0	113m
web-deployment-787c5b74bc-thnkb	1/1	Running	0	113m
web-deployment-787c5b74bc-zn4v5	1/1	Running	0	113m

Wnioski

Kubernetes udostępnia elastyczne metody eksponowania usług na zewnątrz – od bezpośredniego przekierowania portów (port-forwarding) dla pojedynczego poda, aż po abstrakcyjne serwisy (Service) rozkładające ruch na całe wdrożenie. Skalowanie zasobów w klastrze można realizować zarówno imperatywnie (polecenie scale), jak i deklaratywnie (aktualizacja pliku YAML), przy czym podejście deklaratywne jest preferowane ze względu na możliwość trzymania infrastruktury w kodzie (Infrastructure as Code).